But how do you know what signals are relevant, and how do you make the most out of them? Before we get to this topic, let's first step back a bit to set the scene, have a look at common observability use cases, and define roles and tasks.

1.2 Observability use cases

Observability is a means to an end. In other words, when you have a certain challenge or task at hand you want to address, observability supports you in achieving said task faster or managing said challenge more effectively. Let's have a look at common use cases now and see what kind of requirements arise from them:

- *Understanding the impact of code changes*—As a developer, you often add a new feature or fix bugs in your code base. How do you understand the impact of these code changes? What are the relevant data points you need to assess the (potentially negative) effects, such as slower execution or more resource usage?
- *Understanding third-party dependencies*—As a developer, you may use things that are outside of your control—for example, external APIs (payment, location services, etc.). How do you know they are available, healthy, and performing as they should?
- *Measuring user experience (UX)*—As a developer, site reliability engineer (SRE), or operator, you want to make sure your app or service is responsive and reliable. How and where do you measure this?
- *Tracking health and performance*—As an operator, you want to be able to understand the health of your service or app, which can include the overall uptime, the response time distribution, and the parties impacted by a partial or full outage (e.g., paying customers, free-tier users, or canary accounts).
- *Blast radius exploration*—Just like health and UX, on a more fundamental level, you may wish to understand the blast radius for any of your direct apps or services and the underlying platform (e.g., a deployment running in a Kubernetes cluster). How can you tell whether an error is in your app or in the Kubernetes data plane or underlying VM?
- *Optimizing a service*—As a developer, you want to carry out optimizations, including performance, resource usage, costs, and response time of your service. How can you measure these?
- *Increasing developer productivity*—As a service owner or engineering manager, you're interested in your developers being productive. Now, a set of big screens and a comfy gaming chair sure help with that, as well as a no-meeting Monday, Tuesday, and Friday. But how can you measure developer productivity around the code produced from a long-term (maintenance) perspective?
- *Auditing access and tracking compliance*—As an operator, you want to keep track of who has access to different services and customer data, for example. How does your system capture both permissions and access, automatically allow you to alert on unauthorized access, and provide you with an audit trail in case of inspection (for regulatory purposes, let's say)?